

Checkpoint Three Report

CIS*4650 | Hisham Issa | 1194466 | hissa01@uoguelph.ca

Introduction

For this checkpoint, I completed the final piece of the C- compiler, which **was code generation**.

The goal was to take the AST from Checkpoint One and the semantic analysis from Checkpoint Two, and produce TM assembly code that actually runs on the TM simulator. After three checkpoints, the compiler now goes all the way from source code to executable output.

The -c flag triggers code generation. If parsing or semantic analysis finds any errors, code generation is skipped entirely. If the program is clean, the compiler produces a .tm file that can be loaded into the TM simulator and run. This report covers how I implemented the code generator, the design decisions I made, the problems I ran into, and what I learned.

Overview of Previous Checkpoints

Checkpoint One built the front end: a JFlex scanner and a CUP parser that produces an abstract syntax tree. I used operator precedence declarations to simplify the grammar, and added error recovery at key synchronization points so the parser could keep going after errors. The AST was designed around a base Absyn class with subclasses for all C- constructs.

Checkpoint Two added a SemanticAnalyzer using the same visitor pattern. It built a symbol table using a HashMap of name-to-list-of-NodeType entries, handled nested scopes with level numbers, and did type checking for all expressions, assignments, function calls, and control flow. Error recovery assigned dummy types when errors occurred to prevent cascading false positives.

TM Architecture and Runtime Environment

The TM simulator has 1024 instruction memory locations, 1024 data memory locations, and eight registers. I used four of them: ac (r0) as the main accumulator, ac1 (r1) as a secondary one

for operations that need two registers, gp (r6) as the global pointer, fp (r5) as the frame pointer, and pc (r7) as the program counter.

The stack grows downward from gp. Each stack frame has the old frame pointer at offset 0 (ofpFO), the return address at -1 (retFO), and parameters and local variables starting at -2 (initFO). The global pointer stays fixed throughout execution while fp tracks the current frame. Global variables and arrays are allocated relative to gp, and local variables are relative to fp.

CodeGenerator Design

I implemented CodeGenerator as a class that implements AbsynVisitor, same as SemanticAnalyzer. The course slides suggested adding fields like funaddr, offset, and nestLevel directly to the AST node classes, but I decided against that. Modifying the AST classes would mean touching code that was already working from Checkpoints One and Two, which felt risky. Instead, I **used internal HashMaps** to track variable offsets, nesting levels, function entry addresses, and array sizes.

The slides also suggested adding a boolean isAddr parameter to every visitor method to distinguish between computing an address versus loading a value. Same problem: that would require changing the visitor interface and every class that implements it. My solution was a **single boolean field computingAddress in the CodeGenerator**. I set it to true before visiting the left-hand side of an assignment, then restore it after. Same result, no interface changes needed.

For scope management, I used a stack of HashMaps. When entering a function, the current variable maps get pushed onto stacks and a fresh scope starts. When the function exits, the saved

maps get popped back. This handles cases like `sort.cm`, where `minloc()` has a local variable `x` that shadows the global array `x`.

Prelude, I/O Routines, and Finale

Every generated program starts with the same standard prelude: load `gp` with the maximum memory address, copy `gp` to `fp`, and clear location 0. Right after the prelude, I emit the `input()` and `output()` routines and backpatch a jump to skip over them during normal execution. Their entry addresses get saved in `inputEntry` and `outputEntry` so call sites can jump to them later.

The finale comes at the very end, after all function declarations have been processed. It pushes the global frame, jumps to `main`, pops the frame, and halts. If no `main` function was found during code generation, an error comment is emitted, and the finale is skipped.

Expressions, Assignments, and Control Flow

Expression evaluation threads the frame offset through as a level parameter. Each visitor method uses `level` as a pointer to the next available temporary slot, passing `level-1` and `level-2` down to subexpressions so they have their own space. For binary operations, the left operand is evaluated and saved to a temp slot, the right operand is evaluated, and then the left gets loaded into `ac1` for the operation.

Comparisons use a subtract-and-branch pattern: subtract the operands, jump to the true case if the condition holds, load false (0), jump over the true case, load true (1). Assignment computes the left-hand side address first using `computingAddress`, saves it to a temp slot, evaluates the right-hand side, then uses an indirect store to write the value to that saved address.

Control structures use backpatching throughout. For a while loop, I save the test location, emit the condition, emit a placeholder jump using `emitSkip`, generate the body, emit the backward

jump to the test, then fill in the placeholder with the end location using emitBackup and emitRestore. If-else works the same way with two placeholder jumps.

Functions and Recursion

Function declarations emit a jump around the body, record the entry point, store the return address at retFO, allocate offsets for parameters and locals, generate the body, and emit a return.

The call sequence pushes the old frame pointer, sets fp to the new frame, loads the return address into ac, and jumps to the function entry. After returning, fp gets restored from ofpFO.

Recursion works naturally since each call gets its own frame. The interesting case was mutual recursion in mutual.cm, which uses a function prototype. When f calls g before g is defined, funcAddresses doesn't have an entry for g yet. My solution was a forwardCalls map: when a function address is unknown at call time, I emit a placeholder instruction with emitSkip and record the location. When g's definition is later processed, I backpatch all the recorded call sites with the correct address.

Nested Blocks

This one caught me off guard. My original visit(CompoundExp) only processed statements, not declarations, because FunctionDec was already pre-scanning the top-level function body for locals. That worked fine until sort.cm, which declares int t inside a while loop. That declaration is in a nested CompoundExp inside the while body, so FunctionDec never saw it. The compiler emitted an error comment and produced garbage code.

The fix was to update visit(CompoundExp) to check exp.decs and allocate frame slots for any variables declared there before processing the statements. Since the level parameter tracks the next available slot, the nested declarations just consume slots below the current level and pass

the updated level down. This works for variables declared inside if and while bodies and is how sort.cm finally started working.

Arrays and Array Parameters

Arrays were the trickiest part of the whole checkpoint. The layout I settled on stores the array size one slot above the base address, so element[0] sits at the base, element[1] at base+1, and so on. For global arrays, the size gets stored at compile time during array declaration. For local arrays, the size is stored at function entry time using a LDC/ST pair. For array parameters, the parameter slot holds the base address of the caller's array, and the size is still at base+1 relative to that address.

Passing arrays between functions required figuring out the right instruction at call sites. When passing a global or local array, you need LDA to get the base address. But when forwarding an array parameter, the slot already holds an address, so you need LD to load that value. Getting this wrong was causing sort.cm to produce wrong sorted output because minloc was reading from the wrong memory locations entirely.

Runtime Bounds Checking

Every array access checks the index before using it. The check loads the base address into ac1, reloads the index into ac, and does two tests. First, if $\text{index} < 0$, output -1000000 and halt. Second, load the size from base+1, compute $\text{index} - \text{size}$, and if that's not negative, output -2000000 and halt. If both checks pass, the element address is computed and used normally.

There were a few bugs in this that took some time to track down. The first was using ADD ac1,ac,0 to copy a register, which actually computes $2 * \text{ac}$ rather than a copy. The correct instruction is LDA ac1,0(ac). The second was getting the jump offsets wrong in the bounds

check: the JGE and JLT instructions need an offset of 3 to skip the LDC, OUT, and HALT, not 2. Both were caught by tracing the output with the TM simulator.

Changes to Previous Checkpoints

The only meaningful change to Checkpoint Two code was fixing mutual recursion support in the SemanticAnalyzer. Previously, visit(FunctionProtoDec) was an empty stub, which meant mutual.cm failed with errors like "'g' is not a function" and "Symbol 'g' is already defined". The fix was to insert a FunctionDec placeholder (with null body) when a prototype is seen, and to update the insert() method to allow the real definition to replace that placeholder without triggering a redefinition error.

Nothing was changed in the scanner, parser, AST node classes, visitor interface, or ShowTreeVisitor. The conservative design decisions I made for code generation paid off here: because I avoided touching the AST nodes, there was nothing to break.

Testing

I tested incrementally, same as the previous checkpoints. I started with an empty main function, then added simple assignments, then arithmetic, control flow, function calls, recursion, and finally arrays. At each step I verified the generated .tm file by hand and ran it on the TM simulator.

The five provided test programs all produce correct output: fac.cm computes factorials, gcd.cm computes GCD using recursion, booltest.cm computes factorials using a boolean variable, sort.cm sorts 10 integers using selection sort with mutual recursion via minloc, and mutual.cm tests mutual recursion with a function prototype. All five were verified on both my Mac and the Linux server.

For the required ten test files: 1.cm through 3.cm test clean compilation at increasing complexity (basic arithmetic, recursion with booleans, and array reversal). 4.cm through 6.cm test syntax and semantic error detection. 7.cm and 8.cm test runtime bounds checking above and below. 9.cm and 0.cm are stress tests with multiple mixed errors. All ten behave as expected on the Linux server.

Limitations

Local arrays declared inside nested blocks (inside if or while bodies) aren't supported. Only local arrays declared at the top of a function body work. This is because the nested block handling in `visit(CompoundExp)` allocates frame slots for `SimpleDec` variables but not `ArrayDec` declarations.

The temporary slot allocation uses a fixed estimate rather than an exact count, which works for all tested programs but could theoretically overflow for deeply nested expressions. There's also no code optimization, so generated code is larger than it needs to be. The semantic analyzer still doesn't catch using a bare array name in arithmetic (like `a + 1` where `a` is an array), which was a known limitation from Checkpoint Two that wasn't worth fixing at this stage.

Lessons Learned

The TM simulator's trace mode was invaluable for debugging. Every time something went wrong, the right move was to turn on trace, step through the instructions, and compare what was actually happening against what I thought should happen. Trying to debug from the .tm file alone was much slower.

Verifying instruction semantics before writing code saved me from repeating mistakes. The `ADD` register-copy bug happened because I assumed `ADD r,s,0` was a copy, but it's actually `reg[s] +`

reg[0], which is $2 * \text{reg}[s]$ when reg[0] and reg[s] are the same. Checking the instruction table first would have caught this immediately.

Being conservative with changes paid off. Because I kept the AST and visitor interface untouched, I never had to debug problems in already-working checkpoints. The HashMap approach for tracking variable offsets and the single boolean flag for address computation both achieved the same goals as the suggested AST modifications without any of the risk.

Conclusion

The C- compiler is now complete. It scans, parses, type-checks, and generates TM assembly code for valid C- programs, including functions, recursion, mutual recursion, arrays, nested blocks, and runtime bounds checking. All five provided programs produce correct output and the ten required test files all behave as intended.

Code generation was harder than semantic analysis, mostly because errors showed up at runtime rather than compile time and required tracing through assembly to find. But the incremental approach kept things manageable, and the visitor pattern made it straightforward to add code generation alongside the existing analysis passes without disrupting anything that was already working.